A Practical Guide to Deep Q-network

TOPIC -- DEEP Q-NETWORK

-- 01 --

Weight decay A simple form of added regularizer is weight decay, which simply adds an additional error, proportional to the sum of weights (L1 norm) or squared magnitude (L2 norm) of the weight vector, to the error at each node. The level of acceptable model complexity can be reduced by increasing the proportionality constant('alpha' hyperparameter), thus increasing the penalty for large weight vectors. L2 regularization is the most common form of regularization. It can be implemented by penalizing the squared magnitude of all parameters directly in the objective. The L2 regularization has the intuitive interpretation of heavily penalizing peaky weight vectors and preferring diffuse weight vectors. Due to multiplicative interactions between weights and inputs this has the useful property of encouraging the network to use all of its inputs a little rather than some of its inputs a lot. L1 regularization is also common. It makes the weight vectors sparse during optimization. In other words, neurons with L1 regularization end up using only a sparse subset of their most important inputs and become nearly invariant to the noisy inputs. L1 with L2 regularization can be combined; this is called elastic net regularization.

-- 02 --

Time delay neural networks The time delay neural network (TDNN) was introduced in 1987 by Alex Waibel et al. for phoneme recognition and was an early convolutional network exhibiting shift-invariance. A TDNN is a 1-D convolutional neural net where the convolution is performed along the time axis of the data. It is the first CNN utilizing weight sharing in combination with a training by gradient descent, using backpropagation. Thus, while also using a pyramidal structure as in the neocognitron, it performed a global optimization of the weights instead of a local one. TDNNs are convolutional networks that share weights along the temporal dimension. They allow speech signals to be processed time-invariantly. In 1990 Hampshire and Waibel introduced a variant that performs a two-dimensional convolution. Since these TDNNs operated on spectrograms, the resulting phoneme recognition system was invariant to both time and frequency shifts, as with images processed by a neocognitron. TDNNs improved the performance of far-distance speech recognition.

-- 03 --

Weights Each neuron in a neural network computes an output value by applying a specific function to the input values received from the receptive field in the previous layer. The function that is applied to the input values is determined by a vector of weights and a bias (typically real numbers). Learning consists of iteratively adjusting these biases and weights. The vectors of weights and biases are called filters and represent particular features of the input (e.g., a particular shape). A distinguishing feature of CNNs is that many neurons can share the same filter. This reduces the memory footprint because a single bias and a single vector of weights are used across all receptive fields that share that filter, as opposed to each receptive field having its own bias and vector weighting.

-- 04 --

One of the simplest methods to prevent overfitting of a network is to simply stop the training before overfitting has had a chance to occur. It comes with the disadvantage that the learning process is halted.

-- 05 --

Animal behavior detection CNNs have been applied in ecological and behavioral research to automatically detect and quantify animal behavior from visual data, enabling identification of animals, tracking of individuals, estimation of pose, and classification of specific actions such as feeding, and social interactions. Combined with multi-object tracking and temporal modeling, these systems can extract behavioral sequences over extended recordings, reducing reliance on manual annotation and increasing throughput for studies of individual variation, social networks, and collective dynamics.

-- 06 --

Work by Hubel and Wiesel in the 1950s and 1960s showed that cat visual cortices contain neurons that individually respond to small regions of the visual field. Provided the eyes are not moving, the region of visual space within which visual stimuli affect the firing of a single neuron is known as its receptive field. Neighboring cells have similar and overlapping receptive fields. Receptive field size and location varies systematically across the cortex to form a complete map of visual space. The cortex in each hemisphere represents the contralateral visual field. Their 1968 paper identified two basic visual cell types in the brain:

-- 07 --

Another important concept of CNNs is pooling, which is used as a form of non-linear down-sampling. Pooling provides downsampling because it reduces the spatial dimensions (height and width) of the input feature maps while

retaining the most important information. There are several non-linear functions to implement pooling, where max pooling and average pooling are the most common. Pooling aggregates information from small regions of the input creating partitions of the input feature map, typically using a fixed-size window (like 2x2) and applying a stride (often 2) to move the window across the input. Note that without using a stride greater than 1, pooling would not perform downsampling, as it would simply move the pooling window across the input one step at a time, without reducing the size of the feature map. In other words, the stride is what actually causes the downsampling by determining how much the pooling window moves over the input. Intuitively, the exact location of a feature is less important than its rough location relative to other features. This is the idea behind the use of pooling in convolutional neural networks. The pooling layer serves to progressively reduce the spatial size of the representation, to reduce the number of parameters, memory footprint and amount of computation in the network, and hence to also control overfitting. This is known as down-sampling. It is common to periodically insert a pooling layer between successive convolutional layers (each one typically followed by an activation function, such as a ReLU layer) in a CNN architecture. While pooling layers contribute to local translation invariance, they do not provide global translation invariance in a CNN, unless a form of global pooling is used. The pooling layer commonly operates independently on every depth, or slice, of the input and resizes it spatially. A very common form of max pooling is a layer with filters of size 2×2, applied with a stride of 2, which subsamples every depth slice in the input by 2 along both width and height, discarding 75% of the activations: f X , Y (S) = max a , b = 0 1 S 2 X + a , 2 Y + b . {\displaystyle f_{X,Y}(S)=\max _{a,b=0}^{1}S_{2X+a,2Y+b}.}